

LOGIKA PEMROGRAMAN

CASE...OF

& WHILE LOOP

Praktikum ISB 212 – Basisdata | Modul 4

CASE Sederhana

CASE Pencarian

WHILE DO

BREAK & CONTINUE

```
-- SQL Server Logic
DECLARE @grade INT = 85;

SELECT
CASE
    WHEN @grade >= 76
        THEN 'A+'
    WHEN @grade >= 65
        THEN 'A'
    ELSE 'E'
END AS Hasil;

WHILE @i <= 10 BEGIN
    PRINT @i;
    SET @i += 1;
END
```

APA ITU KALIMAT CASE?

CASE adalah ekspresi kondisional di SQL Server yang memungkinkan kita **mengevaluasi kondisi** dan mengembalikan nilai berbeda berdasarkan hasil evaluasi tersebut — mirip seperti **IF-THEN-ELSE** namun digunakan di dalam query SQL.

Analogi Sehari-hari:

Bayangkan seorang kasir supermarket:
"Jika total belanja $\geq$ 500rb $\rightarrow$ diskon 10%
Jika total belanja $\geq$ 200rb $\rightarrow$ diskon 5%
Selain itu $\rightarrow$ tidak ada diskon"

```
-- Contoh analogi kasir dalam SQL

SELECT total,
CASE
    WHEN total >= 500000 THEN '10%'
    WHEN total >= 200000 THEN '5%'
    ELSE 'Tanpa Diskon'
END AS diskon
FROM transaksi;
```

Kapan CASE digunakan?

① Menampilkan Label

Mengubah nilai numerik menjadi teks yang mudah dipahami (misal: nilai 85 $\rightarrow$ 'Lulus')

② Kategorisasi Data

Mengelompokkan data ke dalam kategori berdasarkan rentang nilai atau kondisi tertentu

③ Kondisional di WHERE/ORDER BY

Mengatur urutan atau filter data secara dinamis sesuai kondisi yang berbeda-beda

CASE EKPRESI SEDERHANA

CASE Sederhana membandingkan satu ekspresi dengan sekumpulan nilai. Setiap WHEN hanya dapat menggunakan kesetaraan (=), tidak bisa menggunakan operator > atau <.

SINTAKS UMUM

CASE

ekspresi_input

WHEN nilai1 THEN hasil1

WHEN nilai2 THEN hasil2

WHEN nilaiN THEN hasilN

ELSE hasil_default

END

CONTOH PROGRAM

```
DECLARE @Name VARCHAR(50);
```

```
SET @Name = 'Deepak';
```

```
SELECT
```

```
    CASE @Name
```

```
        WHEN 'Deepak' THEN 'Hello Deepak!'
```

```
        WHEN 'Manoj' THEN 'Hello Manoj!'
```

```
        ELSE 'Siapa kamu?'
```

```
    END AS Pesan;
```

Output:

```
Pesan: Hello Deepak!
```

 Perhatian: CASE Sederhana hanya bisa memeriksa KESETARAAN (=). Untuk perbandingan seperti > atau <, gunakan CASE Pencarian.

CASE EKPRESI PENCARIAN (SEARCHED CASE)

CASE Pencarian tidak menyertakan ekspresi input di awal. Setiap WHEN berisi ekspresi Boolean lengkap, sehingga bisa menggunakan berbagai operator pembandingan.

✗ CASE Sederhana (terbatas)

```
CASE @nilai
  WHEN 90 THEN 'A'  -- Hanya =
  WHEN 80 THEN 'B'  -- Tidak bisa
  -- WHEN > 70 ← ERROR!
END
```

✓ CASE Pencarian (fleksibel)

```
CASE
  WHEN @nilai >= 90 THEN 'A'
  WHEN @nilai >= 80 THEN 'B'
  WHEN @nilai >= 70 THEN 'C'
END
```

📄 Contoh: Grade Mahasiswa

```
SELECT stu_name, marks,
CASE
  WHEN marks >= 76 THEN 'A+'
  WHEN marks >= 65 THEN 'A'
  ELSE 'E'  -- nilai di bawah 65
END AS Grade
```

📄 Output

stu_name	marks	Grade
Deepak	82	A+
Manoj	67	A
Sonal	55	E

💡 Tips: SQL mengevaluasi kondisi WHEN dari atas ke bawah dan berhenti di kondisi pertama yang BENAR. Urutan kondisi sangat penting — letakkan kondisi paling ketat di atas!

CASE DI DALAM WHERE DAN ORDER BY

Ekspresi CASE tidak hanya bisa digunakan di SELECT — ia juga bisa dimasukkan ke dalam klausa WHERE dan ORDER BY untuk filtering dan pengurutan data yang dinamis.

① CASE dalam Klausa WHERE — Menyaring data berdasarkan kondisi yang berbeda

```
SELECT * FROM student_table
WHERE marks >=
  CASE
    WHEN dept = 'Sains' THEN 60
    ELSE 50
  END;
```

Artinya:

- Mahasiswa dept 'Sains'
→ hanya tampilkan jika marks ≥ 60
- Mahasiswa dept lain
→ tampilkan jika marks ≥ 50

② CASE dalam Klausa ORDER BY — Mengurutkan data dengan prioritas berbeda

```
SELECT * FROM student_table
ORDER BY
  CASE dept
    WHEN 'Sains' THEN 1
    WHEN 'Teknik' THEN 2
    ELSE 3
  END;
```

Hasil Urutan:

- Sains → prioritas 1 (Pertama)
- Teknik → prioritas 2 (Kedua)
- Lainnya → prioritas 3 (Terakhir)

💡 CASE dapat digunakan di SELECT, WHERE, ORDER BY, GROUP BY, dan bahkan di dalam fungsi agregat seperti SUM() atau COUNT().

BAGIAN 2

WHILE DO LOOP

01

Konsep & Alur

Cara kerja loop dan evaluasi kondisi

02

Sintaks WHILE

Struktur BEGIN...END di SQL Server

03

Pernyataan BREAK

Keluar dari loop lebih awal

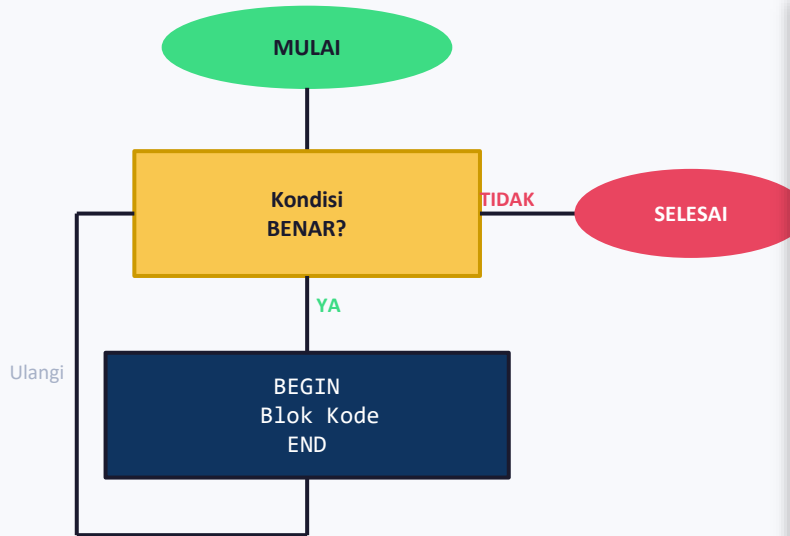
04

Pernyataan CONTINUE

Lewati iterasi saat ini

KONSEP DAN CARA KERJA WHILE LOOP

WHILE adalah pernyataan pengulangan yang terus mengeksekusi blok kode selama kondisi yang diberikan bernilai TRUE (benar).



Langkah-Langkah Eksekusi

1

Cek Kondisi

SQL mengevaluasi kondisi WHILE. Jika FALSE langsung loncat ke setelah END.

2

Jalankan Blok

Jika kondisi TRUE, semua pernyataan dalam BEGIN...END dieksekusi satu per satu.

3

Kembali ke Atas

Setelah END, eksekusi kembali ke baris WHILE untuk mengecek kondisi lagi.

4

Keluar Loop

Loop berhenti ketika kondisi WHILE menjadi FALSE atau ada pernyataan BREAK.

⚠ **Bahaya Infinite Loop:** Jika kondisi WHILE tidak pernah menjadi FALSE (mis. lupa increment counter), program akan berjalan selamanya dan menghabiskan semua resource server!

SINTAKS WHILE LOOP DAN CONTOH PENGGUNAAN

Sintaks + Contoh Counter 1-10

```
-- Sintaks Umum

WHILE kondisi
BEGIN
    -- blok pernyataan
END

-- Contoh: Counter 1 sampai 10
DECLARE @i INT = 1;

WHILE @i <= 10
BEGIN
    PRINT @i;
    SET @i = @i + 1;
    -- ← WAJIB increment!
END
```

Penjelasan Baris demi Baris

```
DECLARE @i INT = 1
    Buat variabel integer, mulai dari nilai 1

WHILE @i <= 10
    Loop selama nilai @i masih ≤ 10

PRINT @i
    Cetak nilai @i saat ini ke output

SET @i = @i + 1
    Tambah @i sebesar 1 (WAJIB, agar loop berhenti)

END
    Kembali ke atas, cek kondisi WHILE lagi
```

Output Program

1	6	← Loop berjalan
2	7	10 kali total
3	8	
4	9	@i dimulai 1,
5	10	berhenti saat
		@i > 10

PERNYATAAN BREAK — KELUAR DARI LOOP

BREAK digunakan untuk menghentikan loop secara paksa sebelum kondisi WHILE menjadi FALSE. Saat SQL menemukan BREAK, eksekusi langsung melompat ke baris setelah END.

Kapan menggunakan BREAK?

Ketika menemukan nilai yang dicari (misal: cari data pertama yang memenuhi syarat)

Ketika terjadi kondisi error atau kondisi khusus yang mengharuskan loop berhenti

Ketika hasil sudah cukup dan tidak perlu memproses sisa iterasi

Contoh: Loop berhenti di ≥ 7

```
DECLARE @i INT = 1;
WHILE @i <= 10
BEGIN
    PRINT @i;
    SET @i = @i + 1;
    IF @i >= 7
        BREAK; -- keluar!
END
```

Output: 1 2 3 4 5 6 (berhenti di sini karena BREAK)

Trace Eksekusi

Iter	@i saat PRINT	@i setelah SET	IF @i >= 7?
1	1	2	Tidak
2	2	3	Tidak
...	...	...	...
6	6	7	YA → BREAK!

 BREAK hanya keluar dari loop terdalam. Jika ada nested loop, BREAK hanya mempengaruhi loop yang memuat BREAK tersebut.

PERNYATAAN CONTINUE — LEWATI ITERASI

CONTINUE melewati sisa pernyataan dalam iterasi saat ini dan langsung kembali ke atas loop untuk mengecek kondisi WHILE lagi — berbeda dengan BREAK yang menghentikan loop sepenuhnya.

BREAK

- Keluar dari loop sepenuhnya
- Eksekusi dilanjutkan setelah END
- Iterasi berikutnya tidak terjadi

CONTINUE

- Lewati sisa kode di iterasi ini
- Kembali ke pengecekan kondisi WHILE
- Loop tetap berjalan seperti biasa

Contoh: Cetak bilangan genap 2–20

```
DECLARE @i INT = 1;
WHILE @i <= 20
BEGIN
    SET @i = @i + 1;
    IF @i % 2 != 0 -- ganjil?
        CONTINUE; -- lewati!
    PRINT @i;     -- hanya genap
END
```

Output: 2 4 6 8 10 12 14 16 18 20

Alur Saat @i = 3 (Ganjil)

- 1** $@i = 1+1 = 2$
→ 2 genap, CONTINUE tidak terpicu → PRINT 2
- 2** $@i = 2+1 = 3$
→ $3 \% 2 = 1 \neq 0$, ganjil → CONTINUE!
- 3** Setelah CONTINUE
→ PRINT dilewati, kembali ke WHILE
- 4** $@i = 3+1 = 4$

Yang Perlu Diingat

CASE Sederhana

- › Gunakan untuk kesetaraan (=) saja
- › Harus ada ekspresi input setelah CASE
- › Tidak bisa memakai operator >, <

CASE Pencarian

- › Tidak ada ekspresi input di awal
- › Setiap WHEN berisi kondisi Boolean
- › Bisa pakai >, <, >=, BETWEEN, dll

WHILE Loop

- › Loop berjalan selama kondisi TRUE
- › Selalu increment counter untuk hindari infinite loop
- › Blok kode dibungkus BEGIN...END

BREAK & CONTINUE

- › BREAK: keluar dari loop sepenuhnya
- › CONTINUE: lewati iterasi saat ini, lanjut loop
- › Keduanya hanya mempengaruhi loop terdalam